

UNIVERSIDAD POLITÉCNICA DE CHIAPAS

PRÁCTICA: DISEÑO DE BASE DE DATOS - REGISTRO DE LICENCIAS

INTEGRANTES:

Ayelen Monserrath Rodriguez Flores - 243681

Fernando Alexis Duran Vazquez -243827

CASO SELECCIONADO:

2-C: Registro de Licencias de Conducir

EXAMEN:

Examen: Diseño de sistemas y Base de Datos - Corte 1

FECHA:

05/02/26

JUSTIFICACIÓN DEL DISEÑO

1. ¿POR QUÉ ESCOGIMOS ESTE CASO?

Seleccionamos el caso 2-C (Registro de Licencias de Conducir) porque:

- Representa un sistema real del gobierno mexicano que ambos conocemos
- Permite implementar conceptos clave de bases de datos relacionales: historial de estados (solicitudes abiertas/aprobadas/rechazadas), relaciones N-N (evaluadores con múltiples especialidades), y manejo de ciclos de vida de entidades (licencias vigentes/reemplazadas)
- Presenta desafíos interesantes como garantizar "una sola licencia vigente por ciudadano" y manejar múltiples intentos de un mismo usuario
- Tiene reglas de negocio claras que se traducen naturalmente a restricciones de base de datos

2. ¿POR QUÉ LO DISEÑAMOS ASÍ?

DECISIONES CLAVE:

a) Surrogate Key para Ciudadano (id_ciudadano) + UNIQUE en CURP:

- CURP como UNIQUE mantiene la integridad del negocio
- ID numérico facilita JOINS y es más eficiente como FK
- Flexibilidad ante posibles cambios en formato de CURP

b) Tabla Puente evaluador_especialidad (relación N-N):

- Refleja la realidad: evaluadores pueden especializarse en ambos exámenes
- Permite consultar fácilmente qué evaluadores pueden aplicar qué exámenes
- Escalable: si agregan nuevos tipos de examen, solo insertamos filas

c) Tablas separadas para examen_medico y examen_teorico:

- Tipos de datos diferentes (resultado vs calificación numérica)
- Lógica de negocio distinta (apto/no apto vs escala 0-100)
- Evita columnas NULL innecesarias
- Simplifica queries específicos por tipo de examen

d) Campo 'estado' en solicitud Y en licencia:

- Permite historial completo de solicitudes (rechazadas, abiertas, aprobadas)
- Estado de licencia independiente facilita renovaciones y reemplazos
- Un ciudadano puede tener múltiples solicitudes pero solo una licencia vigente

e) Restricciones CHECK extensivas:

- Garantizan integridad a nivel de BD (no depende de la aplicación)
- Ejemplo: calificación entre 0-100, vigencia > 0, mayor de edad
- Previenen datos inválidos desde el origen

PUNTO 1: MODELO CONCEPTUAL Y CARDINALIDADES

ESQUEMA RELACIONAL:

- requisito(id PK, codigo, nombre, descripcion, tipo, activo)
- tipo_licencia(id PK, codigo, nombre, vigencia_anios, costo, descripcion, activo)
- tipo_licencia_requisito(tipo_licencia_id PK FK, requisito_id PK FK, obligatorio, orden, fecha_vigencia)
- ciudadano(id PK, curp, nombre, apellido_paterno, apellido_materno, fecha_nacimiento, direccion, telefono, email, fecha_register)
- solicitud(id PK, ciudadano_id FK, tipo_licencia_id FK, fecha_solicitud, estado, fecha_actualizacion, notas)
- evaluador(id PK, nombre, apellido_paterno, apellido_materno, tipo_evaluador, cedula_profesional, activo, fecha_registro)
- examen_teorico(id PK, solicitud_id FK, evaluador_id FK, fecha_examen, calificacion, calificacion_minima, aprobado, observaciones, fecha_registro)
- examen_medico(id PK, solicitud_id FK, evaluador_id FK, fecha_examen, resultado, observaciones, fecha_registro)
- licencia(id PK, numero_licencia, ciudadano_id FK, tipo_licencia_id FK, solicitud_id FK, fecha_emision, fecha_vencimiento, estado, costo_pagado, fecha_actualizacion)

CARDINALIDADES:

- Ciudadano (1) —————< (N) Solicitud
- Tipo_Licencia (1) —————< (N) Solicitud
- Solicitud (1) —————< (1) Examen_medico
- Solicitud (1) —————< (1) Examen_teorico
- Evaluador (1) —————< (N) Examen_medico
- Evaluador (1) —————< (N) Examen_teorico
- Ciudadano (1) —————< (N) Licencia
- Tipo_Licencia (1) —————< (N) Licencia
- Solicitud (1) —————< (0..1) Licencia
- Tipo_Licencia (N) ←————→ (N) Requisito (a través de TIPO_LICENCIA_REQUISITO)

PUNTO 2: RESTRICCIONES IDENTIFICADAS

1. Integridad de Entidad (Primary Keys)

Cada una de las **9 tablas** cuenta con una PRIMARY KEY definida, asegurando que cada registro sea único e identificable.

- **Surrogate Keys:** Uso de id INT AUTO_INCREMENT en casi todas las tablas.
- **Composite Key:** En TIPO_LICENCIA_REQUISITO, la llave primaria es compuesta (tipo_licencia_id, requisito_id), lo que impide duplicar la asignación de un mismo requisito a una licencia.

2. Integridad Referencial (Foreign Keys)

Se garantiza la consistencia entre tablas mediante FOREIGN KEY con reglas de comportamiento específicas:

- **ON DELETE RESTRICT:** Se aplica en relaciones críticas (como Ciudadano → Solicitud) para evitar borrar datos maestros si existen registros dependientes.
- **ON DELETE CASCADE:** Se aplica en EXAMEN_MEDICO y EXAMEN_TEORICO. Si una solicitud se elimina, sus exámenes asociados desaparecen automáticamente para evitar datos huérfanos.
- **ON UPDATE CASCADE:** Permite que si un ID llegara a cambiar, la actualización se propague a todas las tablas relacionadas.

3. Restricciones de Unicidad (UNIQUE)

Evitan la duplicidad de datos de negocio críticos:

- **Identidad:** uq_ciudadano_curp asegura que no haya dos personas con la misma CURP.
- **Documentación:** uq_licencia_numero garantiza que el folio de la licencia sea único.
- **Relación 1:1:** Al marcar solicitud_id como UNIQUE en las tablas de exámenes y licencias, se fuerza por base de datos que **una solicitud solo pueda tener un examen médico, un examen teórico y una sola licencia.**

4. Restricciones de Dominio y Validación (CHECK)

Estas son fundamentales para asegurar que los datos tengan sentido lógico:

- **Formatos:** chk_ciudadano_curp valida que la CURP tenga exactamente 18 caracteres.
- **Rangos Numéricos:** * calificacion en examen teórico debe estar entre 0 y 100.
 - costo y vigencia_anios deben ser mayores a 0.
- **Lógica de Negocio:** chk_licencia_fecha asegura que la fecha de vencimiento sea posterior a la de emisión.
- **Listas Blancas (Enums):** Restringen los valores permitidos en columnas de estado y tipo:
 - **Estado Solicitud:** 'ABIERTA', 'APROBADA', 'RECHAZADA'.
 - **Tipo Evaluador:** 'MEDICO', 'TEORICO', 'AMBOS'.
 - **Resultado Médico:** 'APTO', 'NO_APTO'.

5. Integridad de Columna (NOT NULL / DEFAULT)

- **Campos Obligatorios:** La mayoría de los campos críticos tienen NOT NULL para evitar información incompleta.
- **Valores por Defecto:** Uso de CURRENT_TIMESTAMP para auditoría y estados iniciales automáticos (como estado DEFAULT 'ABIERTA').

6. Restricciones de Cálculo (Generated Columns)

Una restricción avanzada identificada es el uso de **columnas generadas almacenadas**:

- En EXAMEN_TEORICO, la columna aprobado se calcula automáticamente: (calificacion >= calificacion_minima). Esto garantiza que el estatus de aprobación sea siempre verídico respecto a la nota obtenida.

PUNTO 3: QUERYS SQL DE EJEMPLO

Las siguientes consultas demuestran el uso de JOINS, agregaciones, GROUP BY, HAVING, CASE y COALESCE según los requerimientos del negocio.

3.1 CIUDADANOS CON LICENCIA VIGENTE

Propósito: Listar todas las licencias activas con información del titular

Técnicas: INNER JOIN, CASE, filtrado por estado

QUERY 1: Ciudadanos con licencia vigente

-- Ciudadanos con licencia vigente, tipo y fecha de vencimiento

```
SELECT
    c.curp,
    c.nombre,
    c.apellido_paterno,
    c.apellido_materno,
    tl.nombre AS tipo_licencia,
    l.numero_licencia,
    l.fecha_emision,
    l.fecha_vencimiento,
    CASE
        WHEN l.fecha_vencimiento < CURRENT_DATE THEN 'VENCIDA'
        WHEN l.fecha_vencimiento <= CURRENT_DATE + INTERVAL '90 days' THEN
'PRÓXIMA A VENCER'
        ELSE 'VIGENTE'
    END AS situacion
FROM licencia l
INNER JOIN ciudadano c ON l.ciudadano_id = c.id
INNER JOIN tipo_licencia tl ON l.tipo_licencia_id = tl.id
WHERE l.estado = 'vigente'
ORDER BY l.fecha_vencimiento;
```

3.2 PROMEDIO DE CALIFICACIÓN POR TIPO DE LICENCIA

Propósito: Análisis de dificultad de exámenes según tipo de licencia

Técnicas: LEFT JOIN, GROUP BY, HAVING, funciones agregadas, CASE

QUERY 2: Promedio de calificación por tipo de licencia

-- Promedio de calificación del examen teórico por tipo de licencia

```
SELECT
    tl.nombre AS tipo_licencia,
    COUNT(et.id) AS total_exámenes_teoricos,
    ROUND(AVG(et.calificacion), 2) AS promedio_calificacion,
    ROUND(AVG(et.calificacion_minima), 2) AS promedio_calificacion_minima,
    ROUND(MIN(et.calificacion), 2) AS calificacion_minima,
```

```

ROUND(MAX(et.calificacion), 2) AS calificacion_maxima,
COUNT(CASE WHEN et.aprobado = TRUE THEN 1 END) AS aprobados,
COUNT(CASE WHEN et.aprobado = FALSE THEN 1 END) AS reprobados,
ROUND(
    (COUNT(CASE WHEN et.aprobado = TRUE THEN 1 END)::NUMERIC /
    NULLIF(COUNT(et.id), 0)) * 100,
    2
) AS porcentaje_aprobacion
FROM tipo_licencia tl
LEFT JOIN solicitud s ON tl.id = s.tipo_licencia_id
LEFT JOIN examen_teorico et ON s.id = et.solicitud_id
GROUP BY tl.id, tl.nombre
HAVING COUNT(et.id) > 0
ORDER BY promedio_calificacion DESC;

```

3.3 CIUDADANOS CON MÚLTIPLES SOLICITUDES (HISTORIAL)

Propósito: Identificar ciudadanos con intentos múltiples y su historial

Técnicas: GROUP BY, HAVING, STRING_AGG, COUNT con CASE

QUERY 3: Ciudadanos con múltiples solicitudes (historial)

-- Ciudadanos con más de una solicitud y el resultado de cada una

```

SELECT
    c.curp,
    c.nombre,
    c.apellido_paterno,
    c.apellido_materno,
    COUNT(s.id) AS total_solicitudes,
    COUNT(CASE WHEN s.estado = 'aprobada' THEN 1 END) AS aprobadas,
    COUNT(CASE WHEN s.estado = 'rechazada' THEN 1 END) AS rechazadas,
    COUNT(CASE WHEN s.estado = 'abierta' THEN 1 END) AS abiertas,
    STRING_AGG(
        CONCAT(TO_CHAR(s.fecha_solicitud, 'DD/MM/YYYY'), ': ', s.estado),
        ' | ' ORDER BY s.fecha_solicitud
    ) AS historial
FROM ciudadano c
INNER JOIN solicitud s ON c.id = s.ciudadano_id
GROUP BY c.id, c.curp, c.nombre, c.apellido_paterno, c.apellido_materno
HAVING COUNT(s.id) > 1
ORDER BY total_solicitudes DESC;

```

3.4 INGRESO TOTAL POR TIPO DE LICENCIA (ÚLTIMO AÑO)

Propósito: Reporte financiero de ingresos por emisión de licencias

Técnicas: LEFT JOIN, SUM, filtrado temporal con INTERVAL, GROUP BY

QUERY 4: Ingreso total por tipo de licencia (último año)

-- Ingreso total por licencias emitidas en el último año, por tipo

```
SELECT
    tl.nombre AS tipo_licencia,
    tl.costo AS costo_unitario,
    COUNT(l.id) AS licencias_emitidas,
    SUM(l.costo_pagado) AS ingreso_real_total,
    SUM(tl.costo) AS ingreso_esperado_total,
    ROUND(AVG(l.costo_pagado), 2) AS promedio_pagado,
    MIN(l.fecha_emision) AS primera_emision,
    MAX(l.fecha_emision) AS ultima_emision
FROM tipo_licencia tl
LEFT JOIN licencia l ON tl.id = l.tipo_licencia_id
    AND l.fecha_emision >= CURRENT_DATE - INTERVAL '1 year'
    AND l.fecha_emision <= CURRENT_DATE
GROUP BY tl.id, tl.nombre, tl.costo
ORDER BY ingreso_real_total DESC NULLS LAST;
```


3.5 SOLICITUDES RECHAZADAS CON DETALLES DE EXÁMENES

Propósito: Análisis de causas de rechazo para mejorar procesos

Técnicas: Múltiples LEFT JOINs, COALESCE, CASE anidado

QUERY 5: Solicitudes rechazadas con detalles de ambos exámenes

-- Solicitudes rechazadas con resultados de ambos exámenes para identificar causa

```
SELECT
    s.id AS id_solicitud,
    c.curp,
    c.nombre,
    c.apellido_paterno,
    c.apellido_materno,
    tl.nombre AS tipo_licencia_solicitado,
    s.fecha_solicitud,
    -- Examen Médico
    COALESCE(em.resultado, 'NO REALIZADO') AS resultado_medico,
    COALESCE(CONCAT(ev_med.nombre, ' ', ev_med.apellido_paterno), 'N/A') AS
    evaluador_medico,
    COALESCE(TO_CHAR(em.fecha_examen, 'DD/MM/YYYY HH24:MI'), 'N/A') AS
    fecha_medico,
    -- Examen Teórico
    COALESCE(et.calificacion::TEXT, 'NO REALIZADO') AS calificacion_teorico,
    COALESCE(et.calificacion_minima::TEXT, 'N/A') AS calificacion_minima_requerida,
    CASE
        WHEN et.aprobado IS NULL THEN 'NO REALIZADO'
        WHEN et.aprobado = TRUE THEN 'APROBADO'
        ELSE 'REPROBADO'
    END AS resultado_teorico,
    COALESCE(CONCAT(ev_teo.nombre, ' ', ev_teo.apellido_paterno), 'N/A') AS
    evaluador_teorico,
    COALESCE(TO_CHAR(et.fecha_examen, 'DD/MM/YYYY HH24:MI'), 'N/A') AS
    fecha_teorico,
    -- Razón del rechazo
    CASE
        WHEN em.resultado = 'no_apto' AND et.aprobado = FALSE THEN 'AMBOS
    EXÁMENES REPROBADOS'
        WHEN em.resultado = 'no_apto' THEN 'EXAMEN MÉDICO NO APTO'
        WHEN et.aprobado = FALSE THEN 'EXAMEN TEÓRICO REPROBADO'
        WHEN em.resultado IS NULL THEN 'EXAMEN MÉDICO PENDIENTE'
        WHEN et.aprobado IS NULL THEN 'EXAMEN TEÓRICO PENDIENTE'
        ELSE 'CAUSA DESCONOCIDA'
    END AS razon_rechazo
FROM solicitud s
INNER JOIN ciudadano c ON s.ciudadano_id = c.id
INNER JOIN tipo_licencia tl ON s.tipo_licencia_id = tl.id
LEFT JOIN examen_medico em ON s.id = em.solicitud_id
```

```

LEFT JOIN evaluador ev_med ON em.evaluador_id = ev_med.id
LEFT JOIN examen_teorico et ON s.id = et.solicitud_id
LEFT JOIN evaluador ev_teo ON et.evaluador_id = ev_teo.id
WHERE s.estado = 'rechazada'
ORDER BY s.fecha_solicitud DESC;

```

3.6 EVALUADORES Y SUS ESPECIALIDADES (Uso de tabla puente N-N)

Propósito: Verificar qué evaluadores pueden aplicar qué tipos de examen

Técnicas: INNER JOIN con tabla puente, STRING_AGG, GROUP BY

-- Requisitos de cada tipo de licencia (demuestra la relación N:N)

```

SELECT
    tl.codigo AS codigo_tipo,
    tl.nombre AS tipo_licencia,
    tl.vigencia_anios,
    tl.costo,
    COUNT(r.id) AS total_requisitos,
    COUNT(CASE WHEN tlr.obligatorio = TRUE THEN 1 END) AS requisitos_obligatorios,
    COUNT(CASE WHEN tlr.obligatorio = FALSE THEN 1 END) AS requisitos_opcionales,
    STRING_AGG(
        CONCAT(r.nombre, ' (', r.tipo, ')'),
        ', ' ORDER BY tlr.orden
    ) AS lista_requisitos,
    STRING_AGG(
        CASE WHEN tlr.obligatorio = TRUE THEN r.nombre END,
        ', ' ORDER BY tlr.orden
    ) AS requisitos_obligatorios_lista
FROM tipo_licencia tl
LEFT JOIN tipo_licencia_requisito tlr ON tl.id = tlr.tipo_licencia_id
LEFT JOIN requisito r ON tlr.requisito_id = r.id
WHERE tl.activo = TRUE
GROUP BY tl.id, tl.codigo, tl.nombre, tl.vigencia_anios, tl.costo
ORDER BY tl.nombre;

```

PUNTO 4: REPORTES COMO VIEWS

Los siguientes VIEWS implementan los reportes clave que el negocio necesita consultar regularmente. Están optimizados para rendimiento y manejo correcto de casos especiales (como evaluadores con solo un tipo de especialización).

VIEW 1: TASA DE APROBACIÓN POR TIPO DE LICENCIA

Propósito: Monitorear eficiencia del proceso de emisión de licencias

Características:

- Calcula porcentajes de aprobación y rechazo
- Incluye clasificación de relevancia estadística
- Filtra tipos con volumen mínimo (HAVING COUNT >= 5)

-- VIEW 1: Tasa de aprobación por tipo de licencia

-- VIEW 1: Tasa de aprobación por tipo de licencia

CREATE OR REPLACE VIEW vw_tasa_aprobacion_tipo_licencia AS

SELECT

tl.nombre AS tipo_licencia,

COUNT(s.id) AS total_solicitudes,

COUNT(CASE WHEN s.estado = 'aprobada' THEN 1 END) AS solicitudes_aprobadas,

COUNT(CASE WHEN s.estado = 'rechazada' THEN 1 END) AS solicitudes_rechazadas,

COUNT(CASE WHEN s.estado = 'abierta' THEN 1 END) AS solicitudes_abiertas,

ROUND(

(COUNT(CASE WHEN s.estado = 'aprobada' THEN 1 END)::NUMERIC /

NULLIF(COUNT(s.id), 0)) * 100,

2

) AS porcentaje_aprobacion,

ROUND(

(COUNT(CASE WHEN s.estado = 'rechazada' THEN 1 END)::NUMERIC /

NULLIF(COUNT(s.id), 0)) * 100,

2

) AS porcentaje_rechazo,

CASE

WHEN COUNT(s.id) >= 10 THEN 'ESTADÍSTICAMENTE RELEVANTE'

WHEN COUNT(s.id) >= 5 THEN 'MUESTRA LIMITADA'

ELSE 'INSUFICIENTE'

END AS relevancia_estadistica

FROM tipo_licencia tl

LEFT JOIN solicitud s ON tl.id = s.tipo_licencia_id

WHERE tl.activo = TRUE

GROUP BY tl.id, tl.nombre

HAVING COUNT(s.id) >= 5

ORDER BY porcentaje_aprobacion DESC;

Ejemplo de uso:

```
SELECT * FROM vw_tasa_aprobacion_tipo_licencia  
WHERE relevancia_estadistica = 'ESTADÍSTICAMENTE RELEVANTE';
```

VIEW 2: LICENCIAS PRÓXIMAS A VENCER

Propósito: Sistema de alertas para renovaciones

Características:

- Filtra licencias vigentes que vencen en próximos 90 días
- Clasifica por urgencia (30/60/90 días)
- Genera mensajes de alerta personalizados

-- VIEW 2: Licencias próximas a vencer (próximos 90 días)

```
CREATE OR REPLACE VIEW vw_licencias_proximas_vencer AS
```

```
SELECT
```

```
    l.numero_licencia,
```

```
    c.curp,
```

```
    c.nombre,
```

```
    c.apellido_paterno,
```

```
    c.apellido_materno,
```

```
    tl.nombre AS tipo_licencia,
```

```
    l.fecha_emision,
```

```
    l.fecha_vencimiento,
```

```
    l.fecha_vencimiento - CURRENT_DATE AS dias_restantes,
```

```
    CASE
```

```
        WHEN l.fecha_vencimiento - CURRENT_DATE <= 30 THEN 'URGENTE (< 30  
días)'
```

```
        WHEN l.fecha_vencimiento - CURRENT_DATE <= 60 THEN 'ALTA (30-60  
días)'
```

```
        WHEN l.fecha_vencimiento - CURRENT_DATE <= 90 THEN 'MEDIA (60-90  
días)'
```

```
        ELSE 'BAJA (> 90 días)'
```

```
    END AS nivel_urgencia,
```

```
    CONCAT(
```

```
        'Vence en ',
```

```
        l.fecha_vencimiento - CURRENT_DATE,
```

```
        ' días (',
```

```
        TO_CHAR(l.fecha_vencimiento, 'DD/MM/YYYY'),
```

```
        ')'
```

```
    ) AS mensaje_alerta
```

```
FROM licencia l
```

```
INNER JOIN ciudadano c ON l.ciudadano_id = c.id
```

```
INNER JOIN tipo_licencia tl ON l.tipo_licencia_id = tl.id
```

```
WHERE l.estado = 'vigente'
```

```

AND I.fecha_vencimiento <= CURRENT_DATE + INTERVAL '90 days'
AND I.fecha_vencimiento > CURRENT_DATE
ORDER BY I.fecha_vencimiento ASC;

```

Ejemplo de uso:

```

SELECT * FROM vw_licencias_proximas_vencer
WHERE nivel_urgencia IN ('URGENTE (< 30 días)', 'ALTA (30-60 días)');

```

VIEW 3: RENDIMIENTO DE EVALUADORES

- Muestra estadísticas de exámenes aplicados por cada evaluador activo
- Incluye manejo de valores NULL para evaluadores que solo aplican un tipo de examen
- VIEW 3: Rendimiento de evaluadores
- Incluye manejo de valores NULL para evaluadores que solo aplican un tipo de examen.

```

CREATE OR REPLACE VIEW vw_rendimiento_evaluadores AS
SELECT
    ev.id,
    CONCAT(ev.nombre, ' ', ev.apellido_paterno, ' ', COALESCE(ev.apellido_materno, '')) AS
    evaluador,
    ev.cedula_profesional,
    ev.tipo_evaluador,
    -- Exámenes Médicos
    COUNT(DISTINCT em.id) AS total_exámenes_medicos,
    COUNT(DISTINCT CASE WHEN em.resultado = 'apto' THEN em.id END) AS
    medicos_aptos,
    COUNT(DISTINCT CASE WHEN em.resultado = 'no_apto' THEN em.id END) AS
    medicos_no_aptos,
    ROUND(
        (COUNT(DISTINCT CASE WHEN em.resultado = 'apto' THEN em.id END)::NUMERIC
        /
        NULLIF(COUNT(DISTINCT em.id), 0)) * 100,
        2
    ) AS porcentaje_aprobacion_medica,
    -- Exámenes Teóricos
    COUNT(DISTINCT et.id) AS total_exámenes_teoricos,
    COALESCE(ROUND(AVG(et.calificacion), 2), NULL) AS promedio_calificacion_teorica,
    COALESCE(ROUND(AVG(et.calificacion_minima), 2), NULL) AS
    promedio_calificacion_minima_aplicada,

```

```

        COALESCE(ROUND(MIN(et.calificacion), 2), NULL) AS calificacion_minima_teorica,
        COALESCE(ROUND(MAX(et.calificacion), 2), NULL) AS calificacion_maxima_teorica,
        COUNT(DISTINCT CASE WHEN et.aprobado = TRUE THEN et.id END) AS
teoricos_aprobados,
        COUNT(DISTINCT CASE WHEN et.aprobado = FALSE THEN et.id END) AS
teoricos_reprobados,
        ROUND(
            (COUNT(DISTINCT CASE WHEN et.aprobado = TRUE THEN et.id END)::NUMERIC /
            NULLIF(COUNT(DISTINCT et.id), 0)) * 100,
            2
        ) AS porcentaje_aprobacion_teorica,
-- Totales generales
COUNT(DISTINCT em.id) + COUNT(DISTINCT et.id) AS total_exámenes_aplicados,
-- Clasificación de actividad
CASE
    WHEN COUNT(DISTINCT em.id) + COUNT(DISTINCT et.id) = 0 THEN 'INACTIVO'
    WHEN COUNT(DISTINCT em.id) + COUNT(DISTINCT et.id) >= 50 THEN 'MUY
ACTIVO'
    WHEN COUNT(DISTINCT em.id) + COUNT(DISTINCT et.id) >= 20 THEN 'ACTIVO'
    ELSE 'POCO ACTIVO'
END AS nivel_actividad
FROM evaluador ev
LEFT JOIN examen_medico em ON ev.id = em.evaluador_id
LEFT JOIN examen_teorico et ON ev.id = et.evaluador_id
WHERE ev.activo = TRUE
GROUP BY ev.id, ev.nombre, ev.apellido_paterno, ev.apellido_materno,
ev.cedula_profesional, ev.tipo_evaluador
ORDER BY total_exámenes_aplicados DESC;

```

-- Consultar todos los evaluadores

```
SELECT * FROM vw_rendimiento_evaluadores;
```

-- Solo evaluadores muy activos

```
SELECT * FROM vw_rendimiento_evaluadores
```

```
WHERE nivel_actividad = 'MUY ACTIVO';
```

-- Formato amigable para presentación (manejo de NULLs)

```
SELECT
```

```
    evaluador,
```

```
    tipo_evaluador,
```

```
    total_exámenes_medicos,
```

```
    total_exámenes_teoricos,
```

```
    COALESCE(promedio_calificacion_teorica::TEXT, 'N/A') AS promedio_teorico,
```

```
    COALESCE(porcentaje_aprobacion_medica::TEXT || '%', 'N/A') AS aprobacion_medica,
```

```
    COALESCE(porcentaje_aprobacion_teorica::TEXT || '%', 'N/A') AS aprobacion_teorica,
```

```
    nivel_actividad
```

```
FROM vw_rendimiento_evaluadores
```

```
ORDER BY total_exámenes_aplicados DESC;
```